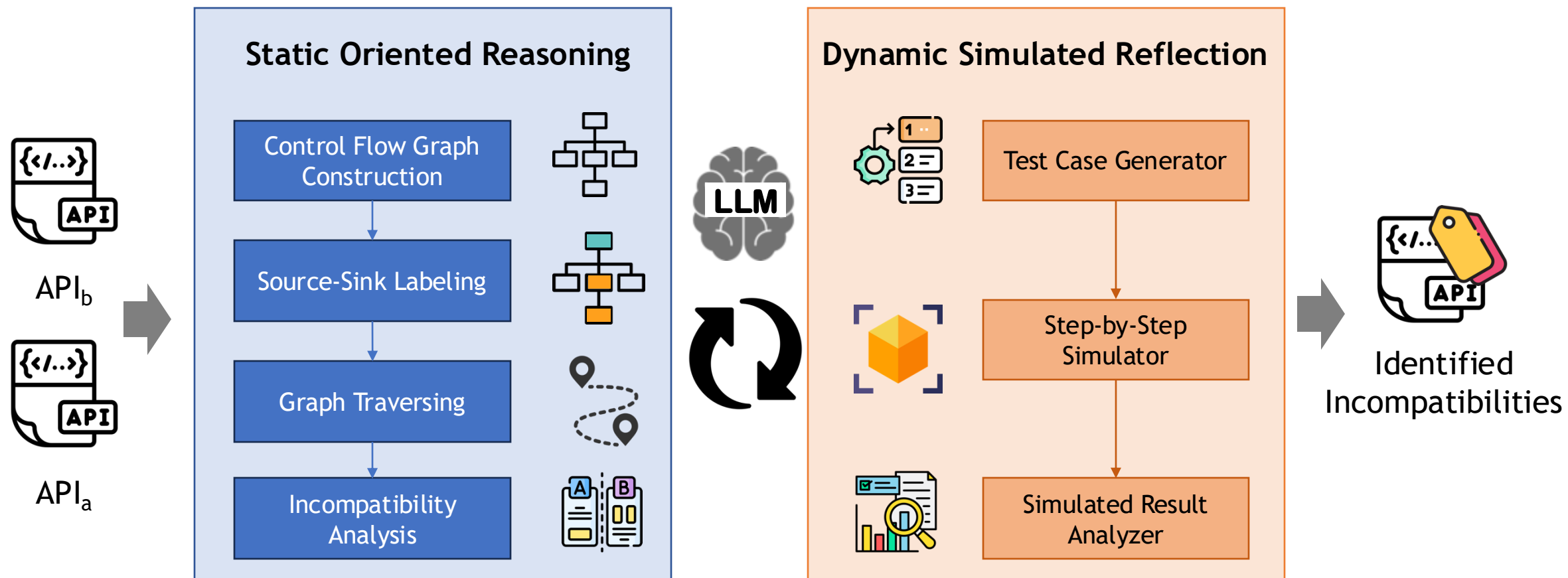




```
1 - def fun(self, amount, n_size = 0, flag = True, **kwargs):
2 + def fun(self, n_dim = 0, flag = False, **kwargs):
3     global GLOBALX
4 -     self.mFlag = flag
5     if flag:
6         return True
7 -     GLOBALX = n_size
8 -     new_var = [0] * n_size
9 -     if kwargs.get('key') and n_size > 0:
10+    GLOBALX = n_dim if n_dim > 0 else None
11+    new_var = [0] * n_dim
12+    if kwargs.get('key for check') and n_dim >= 0:
13        raise TypeError('message')
14    if len(new_var) == 0:
15-        raise ValueError('msg1')
16+        raise LSError('msg2')
17    return new_var
```

- CI1: The required parameter 'amount' from API_b is removed in API_a.
- CI3: The parameter 'n_size' from API_b is renamed to 'n_dim' in API_a.
- CI4: The default value of parameter 'flag' is changed from True in API_b to False in API_a.
- CI9: The assignment statement in API_b is removed in API_a. Consequently, API_a no longer defines the instance attribute mFlag.
- CI8: API_a explicitly assigns None to the global variable when n_dim is less than zero, while API_b may assign an empty instance.
- CI2: The 'key' in kwargs from from API_b is changed to 'key_for_check' in API_a.
- CI6: The raised exception ValueError from API_b is changed to LSError in API_a.
- CI5: The condition that triggers the TypeError exception is changed from Line 9 in API_b to Line 12 in API_a.
- CI7: Given flag as the default input value, API_b returns true at Line 5, while API_a returns new_var.

- Passing deprecated arguments (e.g., amount=1) triggers an “unexpected keyword argument” exception in API_a.
- Passing the argument n_size=10 to API_a triggers an “unexpected keyword argument” exception.
- Without explicitly altering the default value, the same client code may lead to divergent execution logic.
- The use of mFlag in other APIs (i.e., self.mFlag) may trigger an “object has no attribute 'mFlag' ” exception.
- Changes to the value of GLOBALX in other APIs may result in inconsistent behavior.
- Using the same kwargs as in API_b when invoking API_a may trigger an KeyError exception.
- API_a raises a different exception, which may not be properly handled by existing exception-handling logic.
- The assumed exception value changes, which may not be correctly handled by existing exception-handling logic.
- The returned value differs for the same input, leading to inconsistent results.

Change Description

Incompatible API Consequences